

STELLA TECHNOLOGY — INTERNSHIP FINAL PROJECT

Postman Collection Guide

How to import, configure, and test every API endpoint

Base URL: `http://localhost:8080` **Auth:** JWT Bearer Token **Requests:** 20 across 5 folders

Quick Start

1. Open **Postman** (download at postman.com if not installed)
2. Click **Import** (top-left) → drag in `postman_collection.json`
3. The collection *HR Management System* appears in the sidebar
4. Make sure the backend is running on **`http://localhost:8080`**
5. Make sure MongoDB is running on **`localhost:27017`**
6. Run **Login** first (folder 1) → copy the token → paste into collection variables
7. All other requests automatically include the token

Tip: You can also import `openapi.yaml` directly into Postman (Import → select file) to generate a full request collection from the OpenAPI spec automatically.

Collection Variables

Open the collection → click the **Variables** tab. These variables are auto-referenced by all requests.

Variable	Initial Value	How to set
<code>base_url</code>	<code>http://localhost:8080</code>	Change only if backend runs on a different port
<code>jwt_token</code>	<code>empty</code>	Copy from Login response → paste here
<code>employee_id</code>	<code>empty</code>	Copy from any employee response (e.g. <code>EMP-0001</code>)

Variable	Initial Value	How to set
department_id	empty	Copy from any department response (e.g. DEPT-0001)
leave_id	empty	Copy from any leave response (e.g. LVE-0001)
payroll_id	empty	Copy from any payroll response (e.g. PAY-0001)

Folder 1 — Authentication

All authentication endpoints. No token required for these requests.

POST /api/auth/login

Returns a JWT token. Use the response token for all subsequent requests.

```
// Request body (Admin)
{
  "email": "admin@hrms.com",
  "password": "Admin@1234"
}

// Successful response (200 OK)
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "ADMIN",
  "email": "admin@hrms.com"
}
```

After getting the token: copy the value of "token" → open Collection Variables → paste into jwt_token → Save. Login is case-insensitive (e.g. ADMIN@HRMS.COM also works).

GET /api/auth/me

Returns the currently authenticated user's profile.

```
// Response (200 OK)
{
  "email": "admin@hrms.com",
  "role": "ADMIN"
}
```

Folder 2 — Employees

Employee CRUD operations. Requires Admin JWT token unless noted.

GET /api/employees

Paginated list of employees. Accessible by ADMIN and HR.

Query Param	Type	Description
page	int	0-indexed page number (default: 0)
size	int	Results per page (default: 20)
name	String	Filter by name (partial match)
status	String	ACTIVE or INACTIVE
userRole	String	EMPLOYEE, HR, or ADMIN
departmentId	String	Filter by department (e.g. DEPT-0001)

POST /api/employees

Register a new employee. Requires ADMIN.

```
{
  "name": "Jane Smith",
  "email": "jane.smith@company.com",
  "password": "Pass@1234",
  "phone": "+92-301-0000001",
  "gender": "FEMALE",
  "dateOfBirth": "1998-03-12",
  "address": "456 Park Avenue, Lahore",
  "departmentId": "DEPT-0001",
  "designation": "HR Manager",
  "dateOfJoining": "2026-09-01",
  "salary": 70000,
  "allowances": 5000,
  "deductions": 3000,
  "employeeType": "HR",
  "userRole": "HR"
}
```

GET /api/employees/{employeeId}

Get a single employee by their ID (e.g. EMP-0001). Accessible by ADMIN and HR.

PUT /api/employees/{employeeid}

Update an employee's profile. Same request body as POST (all fields optional in update). Requires ADMIN.

DELETE /api/employees/{employeeid}

Delete an employee. Also deletes their user account, leave balance, and leave requests. Requires ADMIN.

```
// Response (200 OK)
{
  "message": "Employee deleted successfully"
}
```

GET /api/employees/{employeeid}/history

Get the promotion/designation/notes history for an employee. Accessible by ADMIN, HR, and the employee themselves.

POST /api/employees/{employeeid}/notes

Add a performance note. Requires ADMIN.

```
{
  "note": "Excellent performance in Q3 2026"
}
```

Folder 3 — Departments

Department management. All require ADMIN.

GET /api/departments

List all departments. Accessible by ADMIN and HR.

POST /api/departments

```
{
  "name": "Engineering",
  "description": "Software development team",
}
```

```
"hrId": "EMP-0001",  
"managerId": "EMP-0002"  
}
```

Required: `hrId` must reference an existing employee with `userRole: "HR"`. One HR per department — if the employee is already assigned to another department the request will return a 400 error.

PUT /api/departments/{departmentId}

Update department details. Same fields as POST, all optional.

DELETE /api/departments/{departmentId}

Delete a department. If the department has employees, they must first be reassigned or a `targetDepartmentId` query param must be provided to move them.

```
DELETE /api/departments/DEPT-0001?targetDepartmentId=DEPT-0002
```

Folder 4 — Leave Management

POST /api/leave/apply

Submit a leave request. Accessible by EMPLOYEE and HR.

```
{  
  "leaveType": "ANNUAL",  
  "startDate": "2026-10-01",  
  "endDate": "2026-10-05",  
  "reason": "Family vacation"  
}
```

Valid leaveType values: `ANNUAL` · `CASUAL` · `SICK` · `UNPAID`

GET /api/leave/my

Get current user's own leave requests. Accessible by EMPLOYEE and HR.

GET /api/leave/my-balance

Get current user's remaining leave balances.

```
// Response (200 OK)
{
  "annualLeaves": 18,
  "casualLeaves": 10,
  "sickLeaves": 8,
  "unpaidLeaves": 5
}
```

GET /api/leave/all

All leave requests org-wide. ADMIN only. Supports query params: `status` , `departmentId` .

GET /api/leave/department

Leave requests for the HR's own department. HR only — automatically scoped to their department by the backend.

POST /api/leave/{leaved}/approve

Approve a pending leave request. HR only.

```
POST /api/leave/LVE-0001/approve
// No request body required
// Response (200 OK): { "message": "Leave approved" }
```

POST /api/leave/{leaved}/reject

Reject a pending leave request. HR only.

GET /api/leave/balances

Look up leave balances for all employees (with optional name filter). ADMIN and HR only.

```
GET /api/leave/balances?name=john
```

Folder 5 — Payroll

POST /api/payroll/generate

Generate a monthly payroll record for an employee. HR only — only employees in the HR's department are accepted.

```
{
  "employeeId": "EMP-0001",
  "month": 9,
  "year": 2026
}
```

Returns 400 if payroll already exists for that employee/month/year combination.

PUT /api/payroll/{payrollId}/status

Mark a payroll record as PAID or UNPAID. HR only.

```
{
  "paymentStatus": "PAID"
}
```

GET /api/payroll/all

All payroll records org-wide. ADMIN only. Supports query params: month , year , departmentId , paymentStatus .

GET /api/payroll/department

Payroll records for the HR's department. HR only — auto-scoped by backend.

GET /api/payroll/my

Current user's own payroll history. Accessible by EMPLOYEE, HR, and ADMIN.

Status Code Reference

Code	Meaning	Common Cause
200 OK	Success	GET, PUT, DELETE succeeded
201 Created	Resource created	POST (register, apply, generate)
400 Bad Request	Validation error	Missing required field, duplicate, insufficient balance

Code	Meaning	Common Cause
401 Unauthorized	Not authenticated	Missing or expired JWT token
403 Forbidden	Not authorized	Role does not have permission for this endpoint
404 Not Found	Resource missing	Invalid employeeId, leaveId, payrollId, etc.

Authentication Header

Every request (except login) must include the `Authorization` header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

The collection handles this automatically — once you set `jwt_token` in collection variables, the header is injected into every request via the collection's pre-request script.